

Malware Analysis & Reverse Engineering

HW2

Course Module: Security & Advanced Virus Analysis

Module 01: Windows API & Structural Analysis

To successfully unpack, reverse engineering, and understand malicious behavior within a Microsoft Windows environment, a deep knowledge of the underlying OS application programming interfaces (APIs) and structural naming standard formatting is necessary.

Theoretical Foundations

- **Hungarian Notation:** Details the explicit resource data architecture, allocating variable naming metrics where prefixes imply structural size/type. (e.g., 'dw' matches a 32-bit unsigned double word integer; 'h' references a generic kernel resource block handle, and 'lp' points directly to a long virtual pointer reference memory layout).
- **System Handles:** Internal abstract references controlled strictly by the Windows Object Manager. When executable malware calls standard system subroutines, it handles non-disclosed data indices to modify processes, threads, or standard network structures.
- **API Interfacing:** Explores how software triggers native kernel routines from User Mode (Ring 3) down to Kernel Mode (Ring 0). Standard security mechanisms often observe these boundaries to map active file read/write mutations.

Task 1: API Analysis within Static Disassemblers (IDA Pro)

Analyze the assembly extraction below representing a standard isolated file creation pipeline found during structural reverse engineering:

```
push 00000002h    ; dwCreationDisposition = CREATE_ALWAYS
push 40000000h    ; dwDesiredAccess = GENERIC_WRITE
push 00000000h    ; dwShareMode = FILE_NO_SHARE
push 00000000h    ; lpSecurityAttributes = NULL
```

```
push 00000002h    ; dwFlagsAndAttributes = FILE_ATTRIBUTE_HIDDEN
push offset FileName ; "psto.exe" (Target structural reference output)
call CreateFileA   ; Executes kernel file allocation block
```

- **Question A:** Explain the operational security consequences of utilizing 'CREATE_ALWAYS' (0x2) compared to 'OPEN_EXISTING' (0x3) when deployed by system injectors or payload drops.
- **Question B:** Identify why the variable 'FileName' is loaded as an offset rather than a direct string block literal in memory stack configurations.

Module 02: Registry Modification & Persistence Mechanisms

Malware authors rely heavily on persistent operating system structures to ensure malicious binaries re-execute seamlessly across machine reboots, power interruptions, or administrative session handovers.

Structural Hive Overview

- **HKLM (HKEY_LOCAL_MACHINE):** Stores comprehensive hardware configuration records, driver layers, and initialization parameters that apply globally to all authentication sequences.
- **HKCU (HKEY_CURRENT_USER):** Holds configurations custom-tailored to the actively logged-in system user context, enabling localized evasion techniques.

AutoStart Extension Points (ASEPs)

The primary Windows architectural keys targeted to achieve continuous code life cycle retention include:

```
HKLM\Software\Microsoft\Windows\CurrentVersion\Run
HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
HKLM\System\CurrentControlSet\Services (For stealth boot loading structures)
```

Task 2: Practical Lab Hunt Utilizing Sysinternals Autoruns

Deploy your designated testing environment virtual machine (Isolated Sandbox) to conduct the following architectural analysis:

- **Step 1:** Identify at least one persistent entry point utilizing the live filtering view inside the Sysinternals Suite.
- **Step 2:** Isolate the cryptographic file hashes linked to the identified binary and document its behavior.
- **Step 3:** Draft an operational detection rule (YARA syntax or Windows Event Log ID structure) capable of blocking unauthorized additions to these specific registry hives.

Module 03: Process Control & Execution Transfer Mechanics

This segment covers advanced concepts surrounding multi-threaded operations, remote memory manipulation, and communication frameworks designed to manage asynchronous threads safely.

Evasion Mechanics

- **DLL Injection:** Allocating virtual memory spaces inside a trusted host environment (e.g., svchost.exe), then copying a dynamic link library link path to force unauthorized execution paths inside native system processes.
- **Thread Hijacking:** Suspending active processing execution lines, capturing context configurations via API, rewriting entry execution addresses, and restoring the thread context to run malicious shellcode.
- **Mutex Coordination:** Mutually Exclusive system flags leveraged by sophisticated binaries to prevent multi-instance overlaps, ensuring single-payload stability.

Task 3: Comparative Architectural Analysis

Construct an analytical report breaking down the tactical execution differences between calling the standard Win32 API layer versus communicating via direct Native APIs (undocumented functions inside ntdll.dll). Your response must cover:

- **1.** API Hooking evasion mechanics implemented by Modern Endpoint Detection and Response (EDR) agents.
- **2.** The structural transition phase shifting down from User Space Ring 3 privileges directly to Ring 0 Kernel Mode operations.